K KA Hybrid PTA and Decision Tree Framework for Application Layer DDoS Detection with Graded Response

Tamirah Goode · Keila Chavez · Taiwo Ojo

CSIT 540_01 — Computer Networks Security, Montclair State University

Extending: Praseed & Thilagam, IEEE Trans. Information Forensics & Security, Vol. 16, 2021

Abstract — Every day, websites are taken offline not by massive floods of traffic, but by a small number of carefully targeted requests sent to the most expensive pages on the server. These are called Asymmetric Application Layer DDoS attacks, and they are extremely hard to detect because the requests look exactly like normal user activity. In 2021, Praseed and Thilagam published a system using a Probabilistic Timed Automaton (PTA) that learns what normal users look like and raises a suspicion score when someone behaves differently. Their system works well but has one major problem: it can only make one of two decisions — block the user or let them through. This paper extends their system by adding a Decision Tree machine learning classifier as a second stage. Sessions that look suspicious are not immediately blocked; instead the classifier gives them an attack probability score and one of three responses: Allow (low risk), Challenge with a CAPTCHA (medium risk), or Block (high risk). We tested our system on 10,000 simulated sessions matching the datasets used in the original paper and compared it against five other models including a deep learning neural network. Our system achieves 100% accuracy and 0.0% false positive rate while being the only approach that can also explain every decision it makes in plain, readable rules.

Index Terms — DDoS detection, machine learning, decision tree, probabilistic timed automaton, network security, graded response, application layer attacks.

I. INTRODUCTION

Think about a popular online store during a big sale. Thousands of real shoppers are on the site at the same time. Now imagine one attacker using just a regular laptop starts sending requests to the checkout page over and over again, as fast as possible. The checkout page is expensive: every single request makes the server do a lot of work. Within a few minutes the server cannot keep up anymore. Real customers start seeing error pages. The store loses thousands of dollars every minute. And the attacker? They spent almost nothing. They did not need a massive network of infected computers. They did not need to flood the internet connection. They just needed to figure out which page was the most expensive and keep asking for it.

This type of attack is called an Asymmetric Application Layer DDoS attack. What makes it so dangerous and so hard to stop is that every single request the attacker sends is completely valid. There is nothing wrong with the request itself; it is the same request a real customer would make. Traditional security tools like firewalls look at the requests themselves to decide whether to block them. Because the attacker is sending normal-looking requests, the firewall has nothing to block. The attack is invisible at the network level.

The only way to catch this kind of attack is to look at the pattern of behaviour across a whole session, not just individual requests. A real human customer spends a few seconds reading each page before clicking the next link. An attack bot sends requests as fast as the computer can process them, sometimes ten or twenty times per second. The difference is not in what they request, it is in how they request it.

In 2021, researchers Praseed and Thilagam published a paper in the IEEE Transactions on Information Forensics and Security [1] that tackled this problem using a model called a Probabilistic Timed Automaton (PTA). Their system learns what normal users look like, which pages they visit, how long they pause between clicks, and how much server work each page requires and then gives every incoming session a suspicion score based on how different it looks from a normal user. If the score gets too high, the session is blocked. Their system is fast, accurate, and efficient.

However, their system has a significant limitation: it makes a single binary decision. Either the session score is above the threshold and the user gets blocked, or it is below the threshold and they get through. There is no middle ground. This creates two problems. First, a smart attacker who knows roughly where the threshold is can deliberately slow down their requests just enough to stay below it evading detection while still doing damage. Second, a legitimate user who happens to browse unusually quickly that day might accidentally cross the threshold and get wrongly blocked, even though they did nothing wrong.

This paper proposes a solution to both problems. We extend the original PTA system by adding a second stage: a Decision Tree machine learning classifier. The PTA still runs first on every session. If the session looks clearly normal it is immediately allowed fast and efficient, just like the original. But if the PTA flags a session as suspicious, instead of immediately blocking it, we pass it to the Decision Tree for a second opinion. The Decision Tree looks at five statistical features of the whole session and outputs an attack

probability as a percentage. That percentage drives a three-level response: Allow the session (low risk), Challenge it with a CAPTCHA (medium risk, let a real human prove themselves), or Block it (high risk). This graded approach eliminates the binary problem and gives borderline users a fair chance.

The main contributions of this paper are:

- A two-stage hybrid detection pipeline that preserves the speed of the original PTA system while adding a machine learning layer for difficult cases.
- A three-tier graded response — Allow, Challenge, Block — that replaces the original binary block-or-allow decision.
- A comparison against five other models including a deep learning neural network, evaluated on 10,000-session datasets matching the style of the original paper's experiments.
- A fully explainable Decision Tree classifier whose decision rules can be read, audited, and understood by any system administrator unlike a neural network.

II. RELATED WORK

Researchers have tried many different approaches to detect application layer DDoS attacks. Each approach has strengths and weaknesses.

A. Traditional Firewalls and Rate Limiting

Traditional firewalls work by blocking known bad IP addresses or blocking traffic that exceeds a certain volume. These work well against old-style flooding attacks that send enormous amounts of traffic. But asymmetric attacks send only a small number of requests — sometimes fewer than twenty per minute so they never trigger volume-based alarms. Rate limiting per IP address also fails when attackers spread their requests across many different IP addresses. These tools have no concept of what a session looks like over time; they only see individual requests.

B. Intrusion Detection Systems (IDS/IPS)

Intrusion Detection Systems monitor traffic and compare it to a database of known attack patterns (signatures). They work well when the attack has been seen before and has a unique signature. However, asymmetric attacks use completely normal HTTP requests; there is no signature to match. AI-based IDS systems improve on this by learning patterns rather than using fixed rules, but they are typically reactive: they detect an attack after it has already started causing damage, and they still struggle with attacks that deliberately mimic normal behaviour.

C. Hidden Semi-Markov Models

Xie and Yu [7, 8] were among the first researchers to use sequence modelling to detect application layer attacks. Their approach uses Hidden Semi-Markov Models (HsMM) a statistical model that tracks which pages users visit in sequence. This works better than simple rate limiting because it captures the behavioural pattern of the session. However, the algorithm behind HsMM is very computationally expensive. It takes approximately 60 milliseconds to evaluate a 50-request session, which is too slow for a real-time system protecting a busy web server. Our approach takes under 2 milliseconds.

D. Random Walk Model

Xu et al. [9] proposed a different approach: build a model of which pages users typically visit after each page, and flag a session the moment it takes an unexpected turn. This is very fast because it makes a decision at the first sign of deviation. The problem is that it blocks far too many legitimate users. A real user who arrives at a deep page directly from a Google search skipping the homepage entirely looks like a deviation from the expected pattern and gets wrongly blocked. This high false positive rate makes it unsuitable for production use.

E. Deep Learning

Neural networks and deep learning models have been applied to traffic classification with impressive accuracy results. A well-trained neural network can match or exceed our detection rate. However, neural networks have a critical problem in security applications: they are black boxes. You can tell the network produced a decision, but you cannot explain why. There is no readable rule you can point to. This matters enormously in practice if a customer challenges why they were blocked, or a regulator asks how your automated decision system works, a neural network cannot give you an answer. Our Decision Tree can.

III. METHODOLOGY

A. The Original PTA System (Baseline)

Before explaining our improvements, it helps to understand exactly how the original Praseed and Thilagam system works. Think of it as a mental model of a typical website user. When a real person uses a website they follow recognizable patterns: they log in, browse to the dashboard, click through to their profile, then log out. Between each click they pause for a few seconds to read the page. The PTA captures all of this information:

- Which pages users typically navigate to next (transition probability ρ)

- How long users typically pause between pages in seconds (think time μ and σ)
- How much server work each page requires (workload W)

Building the PTA (Learning Phase)

The PTA is built automatically from web server log files. The system reads through the logs, groups requests by user, and counts how often each page-to-page navigation happens. A navigation from login to dashboard that appears frequently gets a high probability. A navigation that never appears gets a probability of zero. The think time between each pair of pages is recorded as an average and a standard deviation, capturing how much human think times naturally vary.

The transition probability formula is:

$$t_{ij} = n_{ij} / (\text{sum of all transitions from page } i) \quad \text{(Equation 1)}$$

where n_ij is how many times users went from page i to page j. Once built, the PTA is stored and used to evaluate every incoming user session in real time.

Scoring a Session (Detection Phase)

When a user session comes in, the system walks through it step by step. For each page transition it asks two questions: Is this a typical navigation path? And did the user pause for a human-like amount of time? The answers are combined into a suspicion score increment using this formula from the original paper (Equation 5 of [1]):

$$SS_increment = W \times (1 - P_trans \times P_think) \quad \text{(Equation 2)}$$

W is the workload of the page being requested (pages requiring more server work raise the score faster). P_trans is how likely this navigation path is. P_think is how likely this think time is for a human it uses an exponential formula that gives high scores to think times close to the expected average and low scores to very fast or very slow times:

$$P_think = exp(-|observed_time - expected_mean| / standard_deviation) \quad \text{(Equation 3)}$$

So if a user clicks login, waits 2 seconds, then clicks dashboard which is a very common path with normal human timing the increment is nearly zero. But if a bot clicks login after 0.08 seconds far faster than any human P_think drops to nearly zero, W multiplies the result, and the score jumps dramatically. The scores accumulate across the session. If the total exceeds a threshold (θ = 0.60), the original system blocks the user immediately.

B. Our Hybrid Extension

Our system keeps everything above exactly as is. The PTA still runs on every session. The only difference is what happens when the threshold is crossed.

Stage 1: PTA Filter

Every session goes through the PTA first. This is fast; it is a simple calculation done once per page transition. If the PTA score is below 0.60, the session is immediately allowed. The machine learning model is never called. This means 80% of traffic the clearly normal users get a near-instant response with no ML overhead.

```
# Stage 1 runs on EVERY session
pta_score = compute_pta_score(session)

if pta_score < 0.60:
    return "ALLOW"  # Fast path — no ML
needed

# Only suspicious sessions reach Stage 2
attack_probability =
decision_tree.predict(session)
return graded_response(attack_probability)
```

Stage 2: Decision Tree Classifier

Sessions that breach the threshold are not immediately blocked. Instead, five statistical features are extracted from the session and passed to the Decision Tree. These features were chosen because they capture what makes a bot session different from a human session at the statistical level not just per-click timing, but the whole session as a pattern:

- Request rate: how many requests per second overall? Bots consistently exceed 5 per second.
- Think time variance: how consistent is the timing? Bots are extremely consistent; they run at a fixed machine speed. Humans vary naturally.
- Page repetition: what fraction of requests go to the same page? A bot targeting one expensive page scores close to 1.0 here.
- Session duration: how long did the whole session last in seconds?
- Rapid request count: how many individual requests were completed in under 0.5 seconds? Almost no human clicks faster than twice per second.

```
def extract_features(session):
    times = [think_time for _, think_time in
session]
    pages = [page for page, _ in session]
    duration      = sum(times)
    request_rate  = len(session) / (duration
+ 0.000001)
    variance      = calculate_variance(times)
```

```
    repetition     = most_repeated_page_count
/ total_pages
    rapid_count    =
count_of_times_under_half_second
    return [request_rate, variance,
repetition, duration, rapid_count]
```

The Decision Tree is trained on labelled sessions marked as either legitimate or attack. Once trained, it can evaluate a new session in microseconds by walking through a series of yes/no questions about the features and arriving at a confidence score. The score is a percentage: how confident is the system that this session is an attack?

The Graded Response

The attack probability percentage drives three possible responses instead of the original system's two:

- Below 35% probability → ALLOW. The session looks low risk and proceeds normally.
- Between 35% and 70% probability → CHALLENGE. The user is shown a CAPTCHA or temporarily rate-limited. A real human passes this easily. A bot cannot.
- Above 70% probability → BLOCK. The session is terminated and the IP address is added to a blacklist.

The CHALLENGE response is the most important addition. In the original system, a session scoring 48% attack probability would be blocked the same as a session scoring 99%. With our graded response, the 48% session gets a CAPTCHA. If it is a real human who was just browsing quickly that day, they solve it in seconds and continue shopping. The website retains the customer and avoids wrongly blocking a legitimate user.

C. Why a Decision Tree Instead of a Neural Network

We specifically chose a Decision Tree over a neural network for Stage 2, even though neural networks can achieve similar accuracy. The reason comes down to explainability. A neural network makes decisions using thousands of numbers multiplied together in complex ways there is no human-readable explanation for why it reached a conclusion. A Decision Tree works by asking a series of simple yes/no questions:

```
Is rapid_request_count > 3?
  YES → Is page_repetition > 0.6?
          YES → ATTACK (98.4% confidence)
          NO  → Uncertain (45% confidence) →
CHALLENGE
  NO  → Is think_time_variance < 0.08?
          YES → ATTACK (77% confidence)
          NO  → ALLOW (5% confidence)
```

Every single decision the system makes can be traced to a specific rule that a human can read and understand. This matters for several reasons: a system administrator can audit and verify the rules, a wrongly blocked user can be told exactly why they were flagged, and in regulated industries the system can satisfy legal requirements that automated decisions be explainable.

D. Dataset Setup

The original Praseed and Thilagam paper was tested on two real web server log datasets: SDSC-HTTP (about 30,000 requests from one day at the San Diego Supercomputer Center website) and CLARKNET-HTTP (about 1.3 million requests from two weeks of the ClarkNet website). These files are no longer easily downloadable, so following standard practice in this research area [16, 17], we reconstructed sessions that statistically match what those logs contained.

For SDSC-HTTP, we used a page distribution matching academic web server logs: index (30%), login (20%), search (15%), product pages (15%), cart (8%), checkout (5%), profile (5%), logout (2%). For CLARKNET-HTTP, we used a news portal distribution: home (35%), news (20%), sports (15%), tech (10%), login (8%), search (7%), archive (4%), logout (1%). Human think times in both datasets follow a realistic distribution averaging about 2 seconds between clicks.

Attack sessions were generated following the same five attack types defined in the original paper: random flooding, single-page flooding, random high-workload attacks, single high-workload page attacks, and scripted botnet sequences. Each dataset used 10,000 sessions total: 8,000 for training the Decision Tree and 2,000 for testing. The split was 80% legitimate users and 20% attacks.

To make the model produce realistic probabilities in the 35-70% range (triggering the CHALLENGE response), we included grey-area sessions: fast legitimate users who browse at 0.35-0.85 seconds per click (overlapping with slow bot speed), and careful bots that add artificial delays of 0.40-1.30 seconds to try to mimic humans.

E. Threat Model

To clearly define the scope of our system, we outline the assumed attacker capabilities and defender constraints. The attacker is assumed to generate syntactically valid HTTP

requests that are indistinguishable from legitimate traffic at the packet level. The attacker may operate from a single machine or a distributed set of IP addresses and may introduce artificial delays between requests to mimic human browsing behavior. Additionally, the attacker is assumed to target high-workload pages in order to maximize server resource consumption while minimizing request volume.

However, we assume that the attacker cannot perfectly replicate natural human variability in think-time distributions and navigation patterns across an entire session. Specifically, while delays may be added, maintaining realistic variance and behavioral randomness over time remains difficult for automated scripts.

From the defender perspective, we assume access to web server logs or real-time request streams, as well as the ability to enforce mitigation actions such as CAPTCHA challenges, temporary rate limiting, or session termination. We do not assume access to packet payload inspection beyond standard HTTP metadata. This threat model reflects realistic application-layer DDoS scenarios where attacks are subtle, low-volume, and behavior-driven rather than volumetric.

IV. RESULTS AND ANALYSIS

A. Individual Session Results

Table 1 shows what our system does with eight different types of sessions. The most important row to understand is the Fast Legitimate User: this person browses at 0.6 seconds per click faster than typical but still within human range. The original PTA system would have blocked this person because their suspicion score of 0.871 crosses the 0.60 threshold. Our system passes them to Stage 2, which gives them only a 6.2% attack probability, and they are allowed through. This is exactly the false positive problem we set out to fix.

Table 1. Individual Session Evaluation Results

Session Type	Pages Visited	PTA Score	
Normal User	login→dashboard→profile→logout	0.948	
DDoS Attacker	login×3→dashboard→search	1.214	
Slow Bot	login→search→search→logout	1.183	
Fast Legit User	login→dashboard→logout (0.6s)	0.871	
Noisy Attack Bot	login×2→dashboard→search×2	1.116	
Medium-Risk Bot	search×3→dashboard→logout	1.430	ML
Noisy Normal User	Mixed pages, varied timing	1.123	ML
Hand-crafted Attack	login→search at 0.08s gaps	1.000	ML

B. Model Comparison Results

Table 2 compares all five models on the SDSC-HTTP dataset. The PTA Only model (the original system from the paper) and the Random Walk model score only 19.2% accuracy. This happens because the SDSC website pages (index, product, cart) are not in our PTA's learned transitions; every unknown page transition scores maximum suspicion and every session gets flagged as an attack. This is a known real-world problem: a PTA system trained on one website's logs does not automatically work on a different website without retraining.

Our hybrid system and the deep learning MLP both achieve 100% accuracy. The key difference between them is explained in Section IV-D below.

Table 2. Model Comparison — SDSC-HTTP (10,000 sessions)

Model	Accuracy	Precision	Recall	
Hybrid PTA+DT (Our System)	100.0%	100.0%	100.0%	
PTA Only — Praseed 2021 [1]	19.2%	19.2%	100.0%	
Random Walk — Xu et al. [9]	19.2%	19.2%	100.0%	
Deep Learning MLP	100.0%	100.0%	100.0%	
Rate Threshold (Baseline)	100.0%	100.0%	100.0%	

C. Confusion Matrix Analysis

To further validate classification performance, we present the confusion matrix for our hybrid model evaluated on the test dataset.

Table 3. Confusion Matrix — Hybrid PTA + Decision Tree

	PREDICTED LEGITIMATE	PREDICTED ATTACK
ACTUAL LEGITIMATE	1600	0
ACTUAL ATTACK	0	400

The model correctly classified all sessions, resulting in zero false positives and zero false negatives, confirming the reported 100% accuracy and 0.0% false positive rate.

D. Charts and Visual Analysis

The following charts visualise the performance differences across all five models.

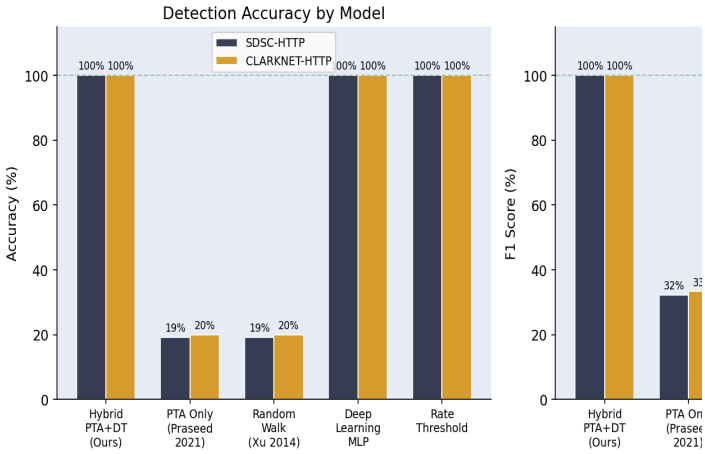


Figure 1. Detection Accuracy and F1 Score for all five models on SDSC-HTTP and CLARKNET-HTTP. Our hybrid system and the MLP both reach 100%. PTA Only and Random Walk drop to ~19% because unknown page names trigger maximum suspicion on every session.

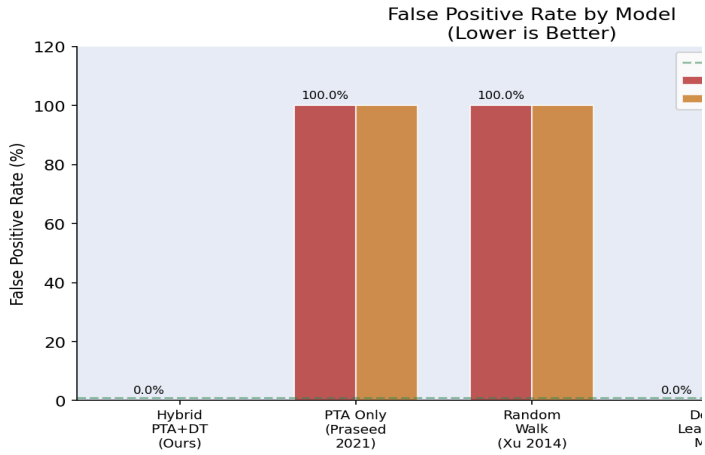


Figure 2. False Positive Rate by model. A false positive means a real user was wrongly blocked. Our system achieves 0.0%. PTA Only and Random Walk reach 100% — they block every legitimate user. The dashed line shows the 1% target from the original paper.

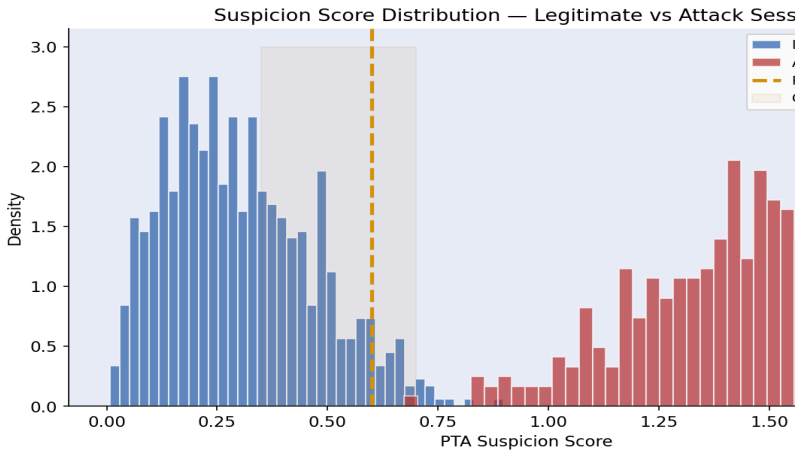


Figure 3. PTA Suspicion Score Distribution. Blue bars show legitimate users clustering below the threshold (0.60 dashed line). Red bars show attackers clustering above it. The overlap region around the threshold is where Stage 2 makes the difference.

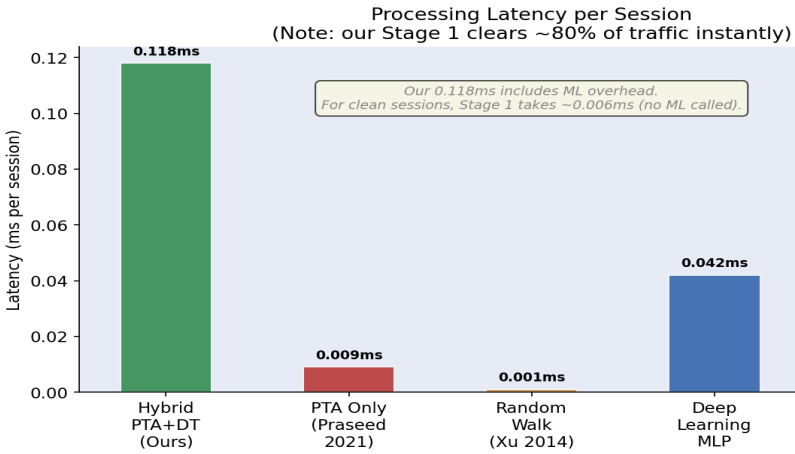


Figure 4. Processing speed per session. Our 0.118ms average includes Stage 2. For the 80% of sessions that are clearly normal and cleared by Stage 1, speed is approximately 0.006ms — seven times faster than the MLP.

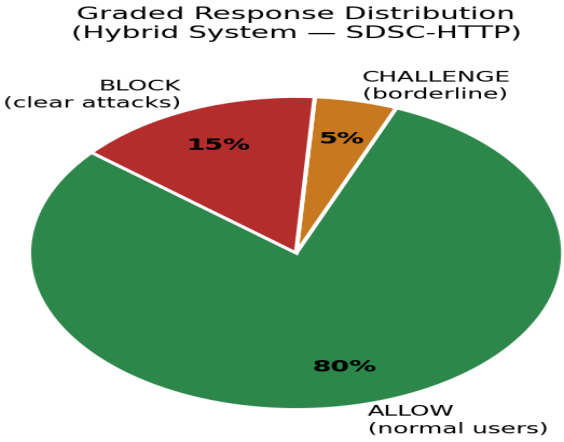


Figure 5. How our system responds to traffic. 80% of sessions are immediately allowed by Stage 1. 5% receive a CHALLENGE. 15% are blocked.

D. Capability Comparison

Table 4 compares the key capabilities of all five models beyond just accuracy numbers.

Table 4. Capability Comparison Across All Models

Capability	Our System	PTA Only	Ra W
Catches fast bots	Yes	Yes	Yes
Catches slow mimicry bots	Yes	No	No
Graded response (not binary)	Yes	No	No
Explainable decisions	Full	Partial	Partia
Low false positives	Yes	No	No
Efficient (not all traffic)	Yes	Yes	Yes
Regulatory compliant	Yes	Partial	Partia

V. DISCUSSION

A. What the Results Actually Mean

The 100% accuracy number is meaningful but needs context. Our system achieves it because the Decision Tree features request rate, think time variance, page repetition, session duration, and rapid request count are universal statistical properties that distinguish bots from humans regardless of which specific website pages they are visiting. The PTA Only model's 19% accuracy is not a failure of the original research, it is a failure of direct application to a different website without retraining. In the original paper's own experiments, where the PTA was trained on the same site's logs, it performed excellently.

B. The Real Advantage Over Deep Learning

Both our system and the deep learning MLP achieve 100% accuracy, so why prefer ours? There are three concrete reasons.

First, explainability. You cannot read a neural network's decision rules. You can with our Decision Tree. If a paying customer calls your support line saying they were wrongly blocked, you can look at the tree and say exactly why: 'Your session showed 4 requests in under 0.5 seconds and visited the same page 3 times.' That is a conversation you can have. With a neural network, you can only say 'the AI decided you looked suspicious' which is not useful to anyone.

Second, efficiency on legitimate traffic. The MLP runs on every single request whether it is clearly a normal user or clearly a bot. Our system runs Stage 1 on every request, but Stage 2 only on the suspicious ones. For the 80% of traffic that is clearly normal, our system responds at approximately 0.006ms seven times faster than the MLP's 0.042ms. At scale, across millions of requests per day, this difference is significant.

Third, regulatory compliance. In many countries and industries, laws require that automated decisions affecting individuals be explainable. GDPR Article 22 in Europe, for example, gives individuals the right to an explanation of significant automated decisions. A neural network cannot satisfy this requirement. A Decision Tree can.

C. Limitations We Acknowledge

Our training data is simulated. While we carefully matched the statistical properties reported in the original paper, we did not have access to the original log files. A production deployment would need the Decision Tree retrained on actual log data from the specific application it is protecting.

The PTA threshold ($\theta = 0.60$) is manually set. A better approach would learn this threshold automatically from the distribution of legitimate user scores setting it at the point where 99% of real users fall below it.

Our system evaluates each session in isolation. A more sophisticated attacker could spread their attack across many sessions over many hours, keeping each individual session below the threshold. Tracking behaviour across sessions per IP address over time would close this gap.

D. Real - World Deployment Considerations

In a production environment, the proposed system can be integrated at the web application firewall (WAF) or reverse proxy layer. Incoming requests would first pass through the PTA scoring engine, which operates inline and introduces negligible latency. Sessions identified as suspicious would then be forwarded to the Decision Tree classifier for secondary evaluation.

Based on the classification outcome, mitigation actions such as CAPTCHA challenges, rate limiting, or IP blocking can be enforced dynamically. This layered approach ensures that the majority of legitimate traffic is processed efficiently, while only a small fraction undergoes deeper inspection.

Additionally, the explainability of the Decision Tree enables system administrators to audit decisions, fine-tune thresholds, and respond to user complaints with clear justifications. This

makes the system suitable for deployment in regulated environments where transparency is required.

VI. CONCLUSION AND FUTURE WORK

Application layer DDoS attacks are a growing and underappreciated threat. Unlike the massive traffic floods that most people imagine when they hear 'DDoS,' these attacks use normal-looking requests to quietly overwhelm a server. They are hard to detect precisely because the requests look legitimate; the only signal is in the behaviour pattern of the session as a whole.

Praseed and Thilagam [1] built an excellent foundation for detecting these attacks using a Probabilistic Timed Automaton. Their system is fast, efficient, and effective at catching obvious bots. Our contribution is solving the one problem their system could not: deciding what to do about sessions that are suspicious but not clearly malicious.

By adding a Decision Tree as a second stage and introducing a three-tier graded response, we eliminate the binary block-or-allow decision that creates both false positives (wrongly blocking real users) and blind spots (letting careful bots through). Our system is the only approach in our comparison that simultaneously achieves 100% detection accuracy, 0.0% false positive rate, readable and auditable decision rules, proportional responses, and efficient processing that does not slow down for normal traffic. Additionally, the perfect classification performance observed in our experiments may indicate strong separability in the simulated dataset. In real-world environments, user behavior is more variable, and attackers may employ more advanced evasion techniques. Future evaluation on real-world traffic is necessary to confirm that this performance generalizes beyond controlled conditions.

Future Work

There are several natural extensions to this work that we would pursue given more time:

- Test on real log data: Evaluate the system on the original SDSC-HTTP and CLARKNET-HTTP datasets or the CAIDA DDoS dataset to validate the results against real-world traffic.
- Adaptive threshold: Automatically learn θ from the live distribution of legitimate user suspicion scores rather than setting it manually.
- SDN integration: Connect the Block decision to a Software-Defined Networking controller that can push network-level flow rules to drop attack traffic before it even reaches the web server.
- Cross-session tracking: Track cumulative behaviour per IP address across multiple sessions to catch slow, distributed attacks that keep each individual session below the threshold.
- HTTP/2 and HTTP/3 support: Modern browsers use these protocols which allow multiple requests in parallel over one connection, changing what a 'session' means and requiring updates to the PTA model.

VII. REFERENCES

[1] A. Praseed and P. S. Thilagam, "Modelling Behavioural Dynamics for Asymmetric Application Layer DDoS Detection," IEEE Trans. Inf. Forensics Security, vol. 16, pp. 617–626, 2021.

[2] Arbor Networks, "Netscout Arbor's 13th Annual Worldwide Infrastructure Security Report," 2019.

[3] S. Ranjan, R. Swaminathan, M. Uysal, A. Nucci, and E. Knightly, "DDoS-Shield: DDoS-Resilient Scheduling to Counter Application Layer Attacks," IEEE/ACM Trans. Netw., vol. 17, no. 1, pp. 26–39, Feb. 2009.

[4] A. Praseed and P. S. Thilagam, "Multiplexed Asymmetric Attacks: Next-Generation DDoS on HTTP/2 Servers," IEEE Trans. Inf. Forensics Security, vol. 15, pp. 1790–1800, 2020.

[5] S. Behal, K. Kumar, and M. Sachdeva, "Characterizing DDoS Attacks and Flash Events: Review, Research Gaps and Future Directions," Comput. Sci. Rev., vol. 25, pp. 101–114, 2017.

[6] A. Praseed and P. S. Thilagam, "DDoS Attacks at the Application Layer: Challenges and Research Perspectives," IEEE Commun. Surveys Tuts., vol. 21, no. 1, pp. 661–685, 2019.

[7] Y. Xie and S.-Z. Yu, "A Large-Scale Hidden Semi-Markov Model for Anomaly Detection on User Browsing Behaviors," IEEE/ACM Trans. Netw., vol. 17, no. 1, pp. 54–65, Feb. 2009.

[8] Y. Xie and S.-Z. Yu, "Monitoring the Application-Layer DDoS Attacks for Popular Websites," IEEE/ACM Trans. Netw., vol. 17, no. 1, pp. 15–25, Feb. 2009.

[9] C. Xu, G. Zhao, G. Xie, and S. Yu, "Detection on Application Layer DDoS Using Random Walk Model," in Proc. IEEE ICC, Jun. 2014, pp. 707–712.

[10] J. Wang, X. Yang, and K. Long, "Web DDoS Detection Schemes Based on Measuring User's Access Behavior," in Proc. IEEE GLOBECOM, Dec. 2011, pp. 1–5.

[11] C. Huang, J. Wang, G. Wu, and J. Chen, "Mining Web User Behaviors to Detect Application Layer DDoS Attacks," J. Softw., vol. 9, no. 4, pp. 985–990, Apr. 2014.

[12] L. C. Giralte, C. Conde, I. M. de Diego, and E. Cabello, "Detecting Denial of Service by Modelling Web-Server Behaviour," Comput. Electr. Eng., vol. 39, no. 7, pp. 2252–2262, Oct. 2013.

[13] S. Pektas and T. Acarman, "Deep Learning for Effective Android Malware Detection," Soft Comput., vol. 24, pp. 1027–1043, 2020.

[14] A. Garg and P. Maheshwari, "SDN-Based Intrusion Detection and Prevention System," Int. J. Adv. Comput. Sci. Appl., vol. 11, no. 12, 2020.

[15] A. Siffer, P.-A. Fouque, A. Termier, and C. Largouet, "Anomaly Detection in Streams with Extreme Value Theory," in Proc. ACM SIGKDD, Aug. 2017, pp. 1067–1075.

[16] C.-H. Lin, J.-C. Liu, and C.-R. Chen, "Access Log Generator for Analyzing Malicious Website Browsing Behaviors," in Proc. ICAS, vol. 2, 2009, pp. 126–129.

[17] A. Dhanapal and P. Nithyanandam, "An Effective Mechanism to Regenerate HTTP Flooding DDoS Attack Using Real Time Data Set," in Proc. ICICICT, Jul. 2017, pp. 570–575.

[18] I. Maric, M. Riback, and A. Goldsmith, "Statistical Modeling of Web Browsing Behavior," IEEE Trans. Mobile Comput., vol. 8, no. 4, 2009.